Reinforcement Learning

Hyoung Joon, Kim

August 6, 2026

Contents

Chapter 1

Introduction

This document is a set of notes of algorithms of reinforcement learning and theories of reinforcement learning. Following books were mainly used for studying algorithms and theories.

- [Sut20];
- [Gra20];
- [Zha25];

Part I

Common Theories

Chapter 2

Basic Concepts

2.1 Introduction

In this section, we discuss common concepts that are used frequently in reinforcement learning.

2.2 State, Action, Policy and Reward

2.2.1 State

The **state** is the status of the agent with respect to the environment. The set of all possible states, the **state space**, is denoted as $\mathcal{S}$.

2.2.2 Action

The **action** is what agent can do, when a state is given. By choosing the action and taking it, the agent moves to the next state, which is called **state transition**. The set of all possible actions in state s, **action space** for state s, is denoted as $\mathcal{A}(s)$.

2.2.2.1 Transition Probability

Note that the transition can be stochastic. That is, the same choice of action in the same state may not lead to equivalent next state. We denote this probability with conditional porbability as following:

$$p(s'|s,a) \tag{2.1}$$

where $s', s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$.

2.2.3 Policy

The **policy** tells the agent which actions to take at every state. The policy can be deterministic or stochastic. We denote the probability of taking action a under state s following policy π as $\pi(a|s)$.

2.2.4 Reward

Reward is one of the most unique concept of reinforcement learning. After the execution of action, the agent receives a reward, denoted as r. The agent's goal is to maximize the total reward that the agent receives. Due to this goal, the reward can encourage or discourage the agent's choice of selecting an action under certain state. Again, the reward can be stochastic, too.

One question that follows naturally is: Can't we just pick the actions that gives the most reward at every state? If this was correct, the reinforcement learning may not have existed. Since the reward is just an immediate information of the act, choosing only the action with highest reward is not the correct answer, most of the time. There can be a better choice in the long run.

2.3 Trajectories, Returns and Episodes

2.3.1 Trajectory

The **trajectory** is the footsteps of the agent. More formally, suppose that the agent is on the initial state s_0 and took action a_0. The agent receives reward r_1 from environment and moves to state s_1. Again, the agent take an action a_1, receive reward r_1 and move to state s_2. Repeating this, we get the trajectory of an agent:

$$s_0, a_0, r_1, s_1, a_1, r_2, \ldots, s_{T-1}, a_{T-1}, rT. \tag{2.2}$$

2.3.2 Returns

Given the trajectory $\tau = \{s_0, a_0, r_1, s_1, a_1, r_2, \ldots, s_{T-1}, a_{T-1}, rT\}$, we can calculate the **return** G_t, of the trajectory. The naive definition of the return, called **total rewards** is the sum

$$G_t = \sum_{k=t+1}^{T} r_k. \tag{2.3}$$

The point is that we do not know if the terminal time T is finite or not. If T is not finite, than the cummulative reward 2.3 diverges. For this, we introduce the **discounted return**, defined as

$$G_t = \sum_{k=t+1}^{\infty} \gamma^{k-t-1} r_k \tag{2.4}$$

where $\gamma \in [0, 1]$ is called **discount rate**. This works for for finite trajectory too, if we let $r_k = 0$ for $k > T$.

2.3.3 Episodes

The **episode** is the finite trajectory itself, like 2.2. The episode may stop at the states called **terminal states**. The tasks with episodes are called **episodic tasks**. Unlike episodic tasks, there are **continuing tasks**, which does not ends. These two can be expressed in unified mathematical manner. For this, in the episodic task, we must consider the agent's behavior after the terminal state: to stay in that state forever, or to take action like normal states. In this note, we follow the latter, like [Zha25].

2.4 Markov Decision Process

2.4.1 Definition

The **Markov decision process** (MDP) is the general framework that describes the stochastic dynamic systems. It is defined as following:

Definition 2.4.1.1 (Markov Decision Process)

The **Markov decision process** is the 5-tuple $(\mathcal{S}, \mathcal{A}, p, \mathcal{R}, \gamma)$, where each elements are defined as following:

- $\mathcal{S}$ is the state space, $\mathcal{A}(s)$ is the action space associated with state s, and $\mathcal{R}(s,a)$ is a set of rewards, associated with each state-action pair (s,a).
- $p(s', r|s, a)$ is the dynamics of the model, which is as probability of moving to state s' from s by taking action a, and obtaining reward r. Note that the state transition probability and the reward probability can be decribed with dynamics. See 2.4.1.1
- $\pi(a|s)$ is the policy, a probability of agent choosing action a in state s.
- γ is the discount rate.

with the dynamics satisfying the *Markov property*:

$$p(s_{t+1}|s_t, a_t, s_{t-1}, a_{t-1}, \ldots, s_0, a_0) = p(s_{t+1}|s_t, a_t) \tag{2.5}$$

$$p(r_{t+1}|s_t, a_t, s_{t-1}, a_{t-1}, \ldots, s_0, a_0) = p(r_{t+1}|s_t, a_t) \tag{2.6}$$

where t is the current time step.

Remark 2.4.1.1 (Transition and reward probability as dynamics)

The 4-tuple dynamics can be used to describe transition probability and reward probability as following:

$$p(s'|s,a) = \sum_{r \in \mathcal{R}(s,a)} p(s', r|s, a) \tag{2.7}$$

$$p(r|s,a) = \sum_{s' \in \mathcal{S}} p(s', r|s, a) \tag{2.8}$$

The dynamics can be either stationary or non-stationary. If the dynamics changes over time, it is non-stationary; else, it is called stationary.

Chapter 3

Bellman Equation

3.1 Introduction

TODO: Introduce the Bellman Operator and Bellman Optimality Operator. The **Bellman equation** is the heart of reinforcement learning. All reinforcement learning algorithms are designed to solve this equation.

3.2 Bellman Equation

3.2.1 State-value Function

Before diving into the Bellman equation, we introduce an important concept called the **value of a state**.

Definition 3.2.1.1 (State Value)

Let the agent follows the policy π. The **state value** $v_\pi(s)$ of state s is defined as the expected return:

$$v_\pi(s) = \mathbb{E}[G_t | S_t = s] \tag{3.1}$$

where G_t is the discounted return $G_t = \sum_{k=t+1}^{\infty} \gamma^{k-t-1} R_k$. The function v_π itself is called **state-value function**.

State values are used to evaluate the policy π. Policies that generate greater state values are better. Then, how can we obtain te state values? The answer is to solve the Bellman equation.

3.2.2 Bellman Equation for State-Value Function

The Bellman equation is simply a set of linear equations that describe the relationships of each state values. Bellman equation can be obtained by using the recursive property of return:

$$\begin{aligned} G_t &= R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \cdots && (3.2)\\ &= R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \gamma^2 R_{t+4}) + \cdots && (3.3)\\ &= R_{t+1} + \gamma G_{t+1}. && (3.4) \end{aligned}$$

Substituting equation 3.1 with the recursive realationship, we obtain

$$v_\pi(s) = \mathbb{E}[G_t|S_t = s] \tag{3.5}$$
$$= \mathbb{E}[R_{t+1} + \gamma G_{t+1}|S_t = s] \tag{3.6}$$
$$= \mathbb{E}[R_{t+1}|S_t = s] + \gamma\mathbb{E}[G_{t+1}|S_t = s] \tag{3.7}$$

Using the law of total expectation **??**, we get

$$\mathbb{E}[R_{t+1}|S_t = s] = \sum_{a\in\mathcal{A}(s)} \pi(a|s)\mathbb{E}[R_{t+1}|S_t = s, A_t = a] \tag{3.8}$$
$$= \sum_{a\in\mathcal{A}(s)} \pi(a|s) \sum_{r\in\mathcal{R}(s,a)} p(r|s,a)r \tag{3.9}$$

and

$$\mathbb{E}[G_{t+1}|S_t = s] = \sum_{s'\in\mathcal{S}} \mathbb{E}[G_{t+1}|S_{t+1} = s', S_t = s]p(s'|s) \tag{3.10}$$
$$= \sum_{s'\in\mathcal{S}} \mathbb{E}[G_{t+1}|S_{t+1} = s']p(s'|s) \quad \text{(Markov property)} \tag{3.11}$$
$$= \sum_{s'\in\mathcal{S}} v_\pi(s')p(s'|s) \tag{3.12}$$
$$= \sum_{s'\in\mathcal{S}} v_\pi(s') \sum_{a\in\mathcal{A}(s)} p(s'|s,a). \tag{3.13}$$

The final result is given in the following box:

Theorem 3.2.2.1 (Bellman Equation for State-Value Function)

Let π be a policy, and let v_π be a state-value function. Then, the following Bellman equation is satisfied:

$$v_\pi(s) = \sum_{a\in\mathcal{A}(s)} \pi(a|s) \sum_{s'\in\mathcal{S}} \sum_{r\in\mathcal{R}(s,a)} p(s', r|s, a)\big[r + \gamma v_\pi(s')\big] \tag{3.14}$$

3.2.3 Matrix-Vector form of Bellman Equation

We can reformulate Bellman equation in terms of matrix-vector form. For this, we rewrite the Bellman equation given in Theorem 3.2.2.1 as

$$v_\pi(s) = r_\pi(s) + \gamma \sum_{s'\in\mathcal{S}} p_\pi(s'|s)v_\pi(s'), \tag{3.15}$$

where

$$r_\pi(s) = \sum_{a\in\mathcal{A}(s)} \pi(a|s) \sum_{r\in\mathcal{R}(s,a)} p(r|s,a)r \tag{3.16}$$
$$p_\pi(s'|s) = \sum_{a\in\mathcal{A}(s)} \pi(a|s)p(s'|s,a). \tag{3.17}$$

The r_π and p_π denotes the mean of immediate rewards and probability of transitioning from s to s' under policy π, respectively. Suppose that we indexed all states as s_i with $i = 1, \ldots, n$, where $n = |\mathcal{S}|$. Let

$$v_\pi = \begin{bmatrix} v_\pi(s_1) \\ \vdots \\ v_\pi(s_n) \end{bmatrix}, r_\pi = \begin{bmatrix} r_\pi(s_1) \\ \vdots \\ r_\pi(s_n) \end{bmatrix}, P_\pi = \begin{bmatrix} p_\pi(s_1|s_1) & p_\pi(s_2|s_1) & \ldots & p_\pi(s_n|s_1) \\ p_\pi(s_1|s_2) & p_\pi(s_2|s_2) & \ldots & p_\pi(s_n|s_2) \\ \vdots & \vdots & & \vdots \\ p_\pi(s_1|s_n) & p_\pi(s_2|s_n) & \ldots & p_\pi(s_n|s_n) \end{bmatrix}. \tag{3.18}$$

Then, Theorem 3.2.2.1 can be written in the following form:

Theorem 3.2.3.1 (Matrix-Vector form of Bellman Equation)

Let v_π, r_π and P_π be given as 3.18. Then, following is satisfied:

$$v_\pi = r_\pi + \gamma P_\pi v_\pi \tag{3.19}$$

Remark 3.2.3.1 (Properties of Matrix P_π)

Note that the matrix P_π given in equation 3.18 satisfies following properties:

- All the elements are nonnegative;
- The sum of the values of every row equals 1. Matrix with this property is called a *stochastic matrix*.

3.3 Solving The Bellman Equation

There are two methods in solving the Bellman equation. The closed-form solution, and the iterative solution.

3.3.1 Closed-form Solution

Since we have represented the Bellman equation as the matrix form, solving it is very easy.

Theorem 3.3.1.1 (Closed-form Solution of The Bellman Equation)

Let π be a given policy, and v_π, r_π, P_π be given as 3.18. The solution to the equation $v_\pi = r_\pi + \gamma P_\pi v_\pi$ is given as

$$v_\pi = (I - \gamma P_\pi)^{-1} r_\pi \tag{3.20}$$

Proof. Manipulate the equation as following:

$$v_\pi = r_\pi + \gamma P_\pi v_\pi \tag{3.21}$$
$$v_\pi - \gamma P_\pi v_\pi = r_\pi \tag{3.22}$$
$$(I - \gamma P_\pi) v_\pi = r_\pi. \tag{3.23}$$

Now, what we have to show is that the matrix $I - \gamma P_\pi$ is invertible. We use the fact that A is invertible $\Leftrightarrow$ eigenvalues of A are nonzero. To show this, we use Geshgorin Circle Theorem 14.2.0.1. Note that the i-th Geshgorin circles has a center at $1 - \gamma p(s_i|s_i)$, with radius $\sum_{j \neq i} \gamma p_\pi(s_j|s_i)$. Since P_π is stochastic matrix, we know that $\sum_j \gamma p(s_j|s_i) = \gamma < 1$. That is,

$\sum_{j \neq i} \gamma p_\pi(s_j|s_i) < 1 - \gamma p_\pi(s_i|s_i)$, which means that the i-th Geshgorin circle cannot contain the origin. According to the Geshgorin Circle Theorem, all the eigenvalues cannot be zero. □

TODO: add properties of $I - \gamma P_\pi$.

3.3.2 Iterative Solution

The direct method for solving the Bellman equation is not used very well in practice, since the inverse matrix computation is very costly. Instead of the close-form method, in practice, we use iterative method to solve the Bellman equation.

Theorem 3.3.2.1 (Iterative Solution of The Bellman Equation)

Let π be a policy and v_π, r_π, and P_π be that of 3.18. Then, the solution of 3.2.3.1 equals the limit of following sequence $\{v_k\}$:

$$v_{k+1} = r_\pi + \gamma P_\pi v_k \tag{3.24}$$

Proof. What we have to show is that $v_k \to v_\pi$ as $k \to \infty$. For this, we let $\delta_k = v_k - v_\pi$ and show that $\delta_k \to 0$. Substituting $v_{k+1} = \delta_{k+1} + v_\pi$ and $v_k = \delta_k + v_\pi$ into equation 3.24, we have

$$\delta_{k+1} = \gamma P_\pi \delta_k. \tag{3.25}$$

Since P_π is a stochastic matrix and $0 < \gamma < 1$, we know that $\delta_k \to 0$ as $k \to \infty$. □

3.4 Bellman Equation for Action-Value Function

Now we introduce the **action value**, which denotes the value of an action at a state.

3.4.1 Action Value

The action value is defined as following:

Definition 3.4.1.1 (Action Value)

Let π be a given policy. Then, the **action value** of a state-action pair is defined as

$$q_\pi(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a]. \tag{3.26}$$

Remark 3.4.1.1 (Properties of Action Value)

Action value $q_\pi(s, a)$ has following properties.

- $v_\pi = \sum_{a \in \mathcal{A}(s)} \pi(a|s) q_\pi(s, a)$
- $q_\pi(s, a) = \sum_{r \in \mathcal{R}(s,a)} p(r|s, a) r + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) v_\pi(s')$

Proof.
- Use the law of total expectation **??**.

- Use the result above and 3.2.2.1.

□

3.4.2 Bellman Equation for Action-Value Function

The Bellman equation for action-value function is already in our hand, since actually the Remark 3.4.1.1 is what all we need.

Theorem 3.4.2.1 (Bellman Equation for Action-value Function)

Let π be a given policy, and let $q_\pi(s,a)$ be an action value. Then, the following Bellman equation is satisfied:

$$q_\pi(s,a) = \sum_{r\in\mathcal{R}(s,a)} p(r|s,a)r + \gamma \sum_{s'\in\mathcal{S}} p(s'|s,a) \sum_{a'\in\mathcal{A}(s')} \pi(a'|s')q_\pi(s',a') \tag{3.27}$$

$$= \sum_{r\in\mathcal{R}(s,a)} p(r|s,a)r + \gamma \sum_{s'\in\mathcal{S}} p(s'|s,a)v_\pi(s') \tag{3.28}$$

Moreover, the matrix-vector form is given as

$$q_\pi = \tilde{r} + \gamma \tilde{P}\Pi q_\pi \tag{3.29}$$

where $[q_\pi]_{(s,a)} = q_\pi(s,a)$, $\tilde{r}_{(s,a)} = \sum_{r\in\mathcal{R}(s,a)} p(r|s,a)r$, $[\tilde{P}]_{(s,a),s'} = p(s'|s,a)$ and Π is a block diagonal matrix such that $\Pi_{s',(s',a')} = \pi(s'|a')$ for $a' \in \mathcal{A}(s')$ and other entries are all zero.

Proof. Use the Remark 3.4.1.1. For the matrix-vector form, try to derive it, as we did in Theorem 3.2.3.1. □

3.4.3 Greedy Policy and ε-greedy Policy

Now, since we have learned about action values, we can define several frequently used policies that depends on action values.

Definition 3.4.3.1 (Greedy Policy)

The **greedy policy** always selects the action with highest action value. That is, given an action-value function $q(s,a)$, the greedy policy π is defined as following:

$$\pi(a|s) = \begin{cases} 1 & \text{if } a = \arg\max_{a'} q(s,a) \\ 0 & \text{otherwise} \end{cases} \tag{3.30}$$

Definition 3.4.3.2 (ε-greedy Policy)

The **ε-greedy policy** always selects the greedy action, but with probability ε, it picks action randomly. That is, given an action-value function $q(s,a)$, the ε-greedy policy π is defined as following:

$$\pi(a|s) = \begin{cases} 1-\varepsilon+\frac{\varepsilon}{|\mathcal{A}(s)|} & \text{if } a = \arg\max_{a'} q(s,a) \\ \frac{\varepsilon}{|\mathcal{A}(s)|} & \text{otherwise} \end{cases} \tag{3.31}$$

Chapter 4

Bellman Optimality Equation

4.1 Introduction

This chapter introduces the Bellman optimal equation (BOE), and proves serveral properties of it. This section mainly follows the book [Zha25].

4.2 Optimal Policy

The reinforcement learning algorithms are all designed to find an optimal policy for given enviroment. Then, what is an 'optimal' policy? The definition is given below.

Definition 4.2.0.1 (Optimal Policy and Optimal State Value)

A policy π^* is optimal if and only if $v_\pi^*(s) \geq v_\pi(s)$ for all $s \in \mathcal{S}$ and for any other policy π. The state values of π^* are the optimal state values.

Definition 4.2.0.2 (Better Policy)

A policy π_1 is **better** than a policy π_2 if and only if $v_{\pi_1}(s) \geq v_{\pi_2}(s)$ for all $s \in \mathcal{S}$.

The questions that naturally arise are:

- Existence: Does the optimal policy exists?
- Uniqueness: Is the optimal policy unique?
- Stochasticity: Is the optimal policy stochastic or deterministic?
- Algorithm: How do we obtain the optimal policy and the optimal state values?

4.3 Bellman Optimality Equation

The optimal policy and optimal state values can be analysed via the **Bellman optimality equation** (BOE). The optimal state values satisfies following Bellman optimality equation. This claim in fact has two logically independent statements:

1. The BOE, viewed purely as a fixed-point equation, has a unique solution (Section 4.4);
2. The unique solution of BOE concides with the optimal stat-value function v^*, and the derived greedy policy is optimal (Section 4.5)

Note that the Section 4.4 does not assume that the state-value function is optimal.

Definition 4.3.0.1 (Bellman Optimality Equation)

Let Π be a set of possible policies and let $\gamma \in (0,1)$ be a discount rate. Then, the Bellman Optimality Equation is defined as

$$v(s) = \max_{\pi \in \Pi} \sum_{a \in \mathcal{A}(s)} \pi(a|s) \left(\sum_{r \in \mathcal{R}} p(r|s,a) r + \gamma \sum_{s' \in \mathcal{S}} p(s'|s,a) v(s') \right) \tag{4.1}$$

$$= \max_{\pi \in \Pi} \sum_{a \in \mathcal{A}(s)} \pi(a|s) q(s,a) \tag{4.2}$$

where $v(s), v(s')$ are unkown variables to be solved. Additionally, the matrix-vector form of Bellman optimality equation is given as

$$v = \max_{\pi \in \Pi} (r_\pi + \gamma P_\pi v) \tag{4.3}$$

where r_π and P_π are given as 3.18.

Using the matrix-vector form of BOE, we can express BOE very simply as following:

$$v = f(v) \tag{4.4}$$

where $f(v) = \max_{\pi \in \Pi}(r_\pi + \gamma P_\pi v)$.

4.4 Solving The Bellman Optimality Equation

4.4.1 Contraction Mapping Theorem

We propose a **contraction mapping theorem**, the main tool for solving the BOE. The contraction mapping theorem is a powerful tool for analyzing general nonlinear equations.

Definition 4.4.1.1 (Contraction Mapping)

Let $f : \mathbb{R}^d \to \mathbb{R}^d$. Then, the function f is a **contraction mapping** if there exists $\gamma \in (0,1)$ such that

$$\|f(x_1) - f(x_2)\| \leq \gamma \|x_1 - x_2\| \tag{4.5}$$

for all $x_1, x_2 \in \mathbb{R}^d$.

Now we present the contraction mapping theorem.

Theorem 4.4.1.1 (Contraction Mapping Theorem)

Let $f : \mathbb{R}^n \to \mathbb{R}^n$ be a contraction mapping. Then, the following properties hold for equation $x = f(x)$:

- Existence: There exists a fixed point x^* satisfying $f(x^*) = x^*$;
- Uniqueness: The fixed point x^* is unique;
- Algorithm: The following sequence $\{x_k\}$ converges to the fixed point x^* exponentially, for any initial x_0.

$$x_{k+1} = f(x_k) \tag{4.6}$$

Proof. Part 1. (Convergence) The sequence $\{x_k\}$ defined as $x_{k+1} = f(x_k)$ converges.

We show that the sequence $\{x_k\}$ is a Cauchy sequence; i.e., there always exists an $N \in \mathcal{N}$ such that $\|x_m - x_n\| < \epsilon$ for any $\epsilon > 0$. The Cauchy sequence has a nice property; the Cauchy sequence always converges. This can be proved by using the properties of $\mathcal{R}$, that the $\mathcal{R}$ is a complete set. We won't show this here. The following holds:

$$\begin{aligned}
\|x_{k+1} - x_k\| =& \|f(x_k) - f(x_{k-1})\| && (4.7)\\
\leq& \gamma\|x_k - x_{k-1}\| && (4.8)\\
\leq& \gamma^2\|x_{k-1} - x_{k-2}\| && (4.9)\\
\leq& \gamma^3\|x_{k-2} - x_{k-3}\| && (4.10)\\
&\vdots && (4.11)\\
\leq& \gamma^k\|x_1 - x_0\| && (4.12)
\end{aligned}$$

Using this, we can show that

$$\begin{aligned}
\|x_m - x_n\| &\leq \|x_m - x_{m-1}\| + \|x_{m-1} - x_{m-2}\| + \cdots + \|x_{n+1} - x_n\| && (4.13)\\
&\leq \gamma^{m-1}\|x_1 - x_0\| + \gamma^{m-2}\|x_1 - x_0\| + \cdots + \gamma^n\|x_1 - x_0\| && (4.14)\\
&= \|x_1 - x_0\|(\gamma^{m-1} + \gamma^{m-2} + \cdots + \gamma^n) && (4.15)\\
&= \|x_1 - x_0\|\frac{\gamma^n(1 - \gamma^{m-n})}{1 - \gamma} && (4.16)\\
&\leq \|x_1 - x_0\|\frac{\gamma^n}{1 - \gamma} && (4.17)
\end{aligned}$$

where $m \geq n$, without loss of generality. Since $\gamma \in (0, 1)$, the last line converges to 0. That is, the sequence $\{x_k\}$ is a Cauchy sequence.

Part 2. (Existence) $\lim_{k\to\infty} x_k$ is a solution of fixed-point equation $x = f(x)$.

Let $x^* = \lim_{k\to\infty} x_k$. Then, following holds.

$$\begin{aligned}
\|x^* - f(x^*)\| &\leq \|x^* - x_k\| + \|f(x_k) - f(x^*)\| + \|x_k - f(x_k)\| && (4.18)\\
&\leq \|x^* - x_k\| + \gamma\|x_k - x^*\| + \|x_k - f(x_k)\| && (4.19)
\end{aligned}$$

Using the results of *Part 1* and squeeze theorem, we can show that $x^* = f(x^*)$.

Part 3. (Uniqueness) The fixed-point x^* is unique.

Suppose there is another fixed point x'. Then, $\|x^* - x'\| = \|f(x^*) - f(x')\| \leq \gamma\|x^* - x'\|$. Since $\gamma \in (0, 1)$, the only possibility is $x^* = x'$.

□

4.4.2 Contraction Property of Bellman Optimality Equation

Now, to use the contraction mapping theorem, we prove that the right-hand side of BOE 4.3 is a contraction mapping.

Theorem 4.4.2.1 (Contraction Property of Bellman Optimality Equation)

Let the function f be that of equation 4.3. Then, the following contraction mapping property holds for any $v_1, v_2 \in \mathbb{R}^{|\mathcal{S}|}$:

$$\|f(v_1) - f(v_2)\|_\infty \leq \gamma\|v_1 - v_2\|_\infty \tag{4.20}$$

where $\gamma \in (0, 1)$ is the discount rate, and $\|\|_\infty$ is the maximum norm, which is the maximum absolute value of the elements in a vector.

Proof. Let $v_1, v_2 \in \mathbb{R}^{\mathcal{S}}$, and let $\pi_1^* = \arg\max_\pi (r_\pi + \gamma P_\pi v_1)$, $\pi_2^* = \arg\max_\pi (r_\pi + \gamma P_\pi v_2)$. Then,

$$f(v_1) = \max_\pi (r_\pi + \gamma P_\pi v_1) = r_{\pi_1^*} + \gamma P_{\pi_1^*} v_1 \geq r_{\pi_2^*} + \gamma P_{\pi_2^*} v_1 \tag{4.21}$$

$$f(v_2) = \max_\pi (r_\pi + \gamma P_\pi v_2) = r_{\pi_2^*} + \gamma P_{\pi_2^*} v_2 \geq r_{\pi_1^*} + \gamma P_{\pi_1^*} v_2 \tag{4.22}$$

where $\geq$ is applied elementwise. This result leads to

$$f(v_1) - f(v_2) \leq \gamma P_{\pi_1^*}(v_1 - v_2) \tag{4.23}$$

$$f(v_2) - f(v_1) \leq \gamma P_{\pi_2^*}(v_2 - v_1) \tag{4.24}$$

Let $z = \max\{|\gamma P_{\pi_1^*}(v_1 - v_2)|, |\gamma P_{\pi_2^*}(v_2 - v_1)|\}$, where $\max(\cdot)$ and $|\cdot|$ are all applied elementwise. Then,

$$\|f(v_1) - f(v_2)\|_\infty \leq \|z\|_\infty. \tag{4.25}$$

Now, let z_i be the i-th entry of z, and let $p_i^\top$ and $q_i^\top$ be the i-th row of $P_{\pi_1^*}$ and $P_{\pi_2^*}$, respectively. Then, $z_i = \max\{\gamma|p_i^\top(v_1 - v_2)|, \gamma|q_i^\top(v_1 - v_2)|\}$. Since all the elements of vector p_i is nonnegative and thier sum equals 1,

$$|p_i^\top(v_1 - v_2)| \leq p_i^\top|v_1 - v_2| \leq \|v_1 - v_2\|_\infty. \tag{4.26}$$

where $|\cdot|$ is applied elementwise. Similarly, $|q_i^\top(v_1 - v_2)| \leq \|v_1 - v_2\|_\infty$. Therefore $\|z\|_\infty \leq \gamma\|v_1 - v_2\|_\infty$. Substituting to 4.25,

$$\|f(v_1) - f(v_2)\|_\infty \leq \gamma\|v_1 - v_2\|_\infty. \tag{4.27}$$

□

4.5 Optimal Policy, Optimal State Value and Bellman Optimality Equation

We have a following Theorem of BOE:

Theorem 4.5.0.1 (Solution of the Bellman Optimality Equation)

Let f be as the equation 4.3. Then, the BOE $v = f(v)$ has following properties:

1. There exists a unique solution;
2. The solution can be obtained iteratively by $v_{k+1} = f(v_k)$.

Proof. All the previous Theorems (4.4.1.1, 4.4.2.1) proves this trivially. □

Now, we should prove that the solution of the Bellman optimality equation is really an optimal state-value function, and prove that the greedy policy derived from it is the optimal policy.

Theorem 4.5.0.2 (Optimality of v^* and π^*)

Let v^* be the solution of BOE $v = f(v)$, and let $\pi^* = \arg\max_{\pi \in \Pi}(r_\pi + \gamma P_\pi v^*)$. The, the v^* is an optimal state vaule function, and the π^* is an optimal policy. That is, for any policy π, following holds;

$$v^* = v_{\pi^*} \geq v_\pi \tag{4.28}$$

where v_π is the state value function of π, and $\geq$ is applied elementwise.

Proof. For any policy π, $v_\pi = r_\pi + \gamma P_\pi v_\pi$ holds (Bellman equation). Then,

$$\begin{aligned} v^* &= \max_{\tau \in \Pi}(r_\tau + \gamma P_\tau v^*) && (4.29)\\ &= r_{\pi^*} + \gamma P_{\pi^*} v^* && (4.30)\\ &\geq r_\pi + \gamma P_\pi v^* && (4.31) \end{aligned}$$

and we have

$$v^* - v_\pi \geq \gamma P_\pi (v^* - v_\pi). \tag{4.32}$$

Using this inequality repeatedly, we have $v^* - v_\pi \geq \gamma^n P_\pi^n (v^* - v_\pi)$ for any $n \in \mathbb{N}$. Since $\gamma \in (0, 1)$ and P_π is a nonnegative matrix with all its elements less than or equal to 1, $\lim_{n\to\infty} \gamma^n P_\pi^n (v^* - v_\pi) = 0$. That is, $v^* - v_\pi \geq \lim_{n\to\infty} \gamma^n P_\pi^n (v^* - v_\pi) = 0$. □

The following theorem examines the optimal policy more precisely.

Theorem 4.5.0.3 (Greedy Optimal Policy)

For any $s \in \mathcal{S}$, the following deterministic policy is an optimal policy for solving the BOE.

$$\pi^*(a|s) = \begin{cases} 1, & a = \arg\max_a q^*(a, s) \\ 0, & a \neq \arg\max_a q^*(a, s) \end{cases} \tag{4.33}$$

where $q^*(s, a) = \sum_{r \in \mathcal{R}} p(r|s, a)r + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a)v^*(s')$.

Proof. From Theorem 4.5.0.1 and 4.3.0.1, we see that

$$v^*(s) = \max_{\pi \in \Pi} \sum_{a \in \mathcal{A}} \pi(a|s) q^*(s, a) \tag{4.34}$$

$$\leq \left(\max_{\pi \in \Pi} \sum_{a \in \mathcal{A}} \pi(a|s) \right) \max_{a'} q^*(s, a') \tag{4.35}$$

$$= \max_{a'} q^*(s, a') \tag{4.36}$$

and the given deterministic policy π^* satisfies the equality of inequality 4.35. □

The important truth is that the optimal state value is unique but the optimal policy is not. The Theorem 4.5.0.3 only shows that there always exists at least one deterministic optimal policy. There still can be other optimal policies, too, stochastic or whatever.

4.6 Factors that Influence Optimal Policies

Now, we analyze the factors of BOE that influences optimal policies. The factors are: reward and discount rate.

4.6.1 Reward

Changing reward can lead to slightly different behavior of policies. For example, in the Figure 3.4(a) of [Zha25] (gridworld), one can see that the agent tries to move to the goal even though there are forbidden areas. The forbidden area gives the negative reward, but the cumulative reward is greater than going around the forbidden areas. This is why reward (signal) is important.

4.6.2 Discount Rate

High dicount rate leads to far-sighted agent, and low discount rate leads to short-sighted agent. In the extreme, when γ, the agent only take into account the immediate reward.

4.6.3 Invariance of the Optimal Policy

Changing the reward may lead to different optimal policies, but this is not always the case. If we apply the same affine transformation to all rewards, the optimal policy remains the same.

Theorem 4.6.3.1 (Optimal Policy Invariance)

Let $v^* \in \mathbb{R}^{|\mathcal{S}|}$ be the optimal state value satisfying $v^* = \max_{\pi \in \Pi} (r_\pi + \gamma P_\pi v^*)$. If every reward $r \in \mathcal{R}$ goes through the same affine transformation $\alpha * r + \beta$, where $\alpha, \beta \in \mathbb{R}$ and $\alpha > 0$, the corresponding optimal state value v' is also an affine transformation of v^*:

$$v' = \alpha v^* + \frac{\beta}{1-\gamma} \mathbb{1} \tag{4.37}$$

where $\gamma \in (0, 1)$ is the discount rate and $\mathbb{1} = [1, \ldots, 1]^\top$. Due to this property, the optimal policy stays invariant.

Chapter 5

Stochastic Approximation Theory

5.1 Introduction

This chapter introduces measure-theoretic probability, convergence and several stochastic approximation theorems, which is a theoretic foundation for TD-Learning algorithms.

5.1.1 Prerequisite

The subsections ahead requries a basic knowledge about measure-theoretic probability, which are provided in 13. The probability theory was re-established by the measure theory, which enabled to define all the ambiguous concepts of probability theory rigorously. For comprehensive introduction about measure theory, see [Axl20] and for more information about measure-theoretic probability, see [BZ07].

5.1.2 Robbins-Monro Algorithm

Suppose we want to find the root of the equation

$$g(w) = 0. \tag{5.1}$$

There are many other iterative algorithms for finding the root of a given equation. Then, why do we need stochastic approximation? In real world problems, the expression of function g or its derivative is unknown, and we may even only be able to obtained the noisy observations of g. That is, with the original g and noise η, we can only observe $\tilde{g}(w, \eta) = g(w) + \eta$. The SGD method is one of the most well-known solution to problems like this. But here, we introduce a more general algorithms of stochastic approximation, called **Robbins-Monro algorithm**.

Theorem 5.1.2.1 (Robbins-Monro Algorithm)

Let g be a function and let $\tilde{g}(w, \eta) = g(w) + \eta$ where η is a random noise. Then, the sequence $\{w_k\}$ defined as $w_{k+1} = w_k - a_k \tilde{g}(w_k, \eta_k)$ where $\{a_k\}$ is a sequence of positive coefficients almost surely converges to the root of equation $g(w) = 0$ under following conditions:

1. $0 < c_1 \leq \nabla_w g(w) \leq c_2$ for all w;
2. $\sum_{k=1}^{\infty} a_k = \infty$ and $\sum_{k=1}^{\infty} a_k^2 < \infty$;
3. $\mathbb{E}[\eta_k | \mathcal{H}_k] = 0$ and $\mathbb{E}[\eta_k^2 | \mathcal{H}_k] < \infty$;

where $\mathcal{H}_k = \{w_k, w_{k-1}, \dots\}$.

Proof. TODO: proof of RM algorithm □

Part II

Algorithms

Chapter 6

Dynamic Programming

6.1 Introduction

This chapter discuss and dyamic programming algorithms for finding optimal policies. Since these are dynamic prgramming algorithms, a perfect system model is required; this mean that it is rarely used in practice, but these algorithms important foundations of the model-free algorithms. Three algorithms will be presented;

- Value Iteration: The algorithms suggested by the contraction mapping theorem 4.5.0.1;
- Policy Iteration:
- Trucated Policy Iteration: The unified algorithm which includes value iteration and policy iteration as its special cases.

6.2 Value Iteration

6.2.1 Main Idea

The main idea of **value iteration** algorithms is the algorithms suggested in Theorem 4.5.0.1. In particular, the algorithms is

$$v_{k+1} = \max_{\pi \in \Pi} (r_\pi + \gamma P_\pi v_k), \quad k = 0, 1, 2, \dots \tag{6.1}$$

Breaking down, this iterative algorithm has two steps;

1. *Policy Update*: The algorithm finds a policy π_{k+1} that solves following optimization problem;

$$\arg\max_{\pi} (r_\pi + \gamma P_\pi v_k) \tag{6.2}$$

2. *Value Update*: With the policy π_{k+1}, the algorithm calculates a new state value function v_{k+1};

$$v_{k+1} = r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_k \tag{6.3}$$

and this v_{k+1} is used in next iteration.

6.2.2 Pseudocode

Algorithm 1: Value Iteration Algorithm

Input : $p(s', r|s, a)$ ..... the dynamics of the system
v ..... initial guess

Output: v^*, π^* ..... optimal state value and optimal policy

$\pi \leftarrow$ greedy policy derived from v
repeat
  for $s \in \mathcal{S}$ **do**
    for $a \in \mathcal{A}(s)$ **do**
      $q(s, a) \leftarrow \sum_r p(r|s, a)r + \gamma \sum_{s'} p(s'|s, a)v(s')$
    `// policy update`
    $\pi(a|s) = \begin{cases} 1 & \text{if } a = \arg\max_a q(s, a) \\ 0 & \text{otherwise} \end{cases}$
    `// value update`
    $v(s) = \max_a q(s, a)$
until convergence of v
return v, π

Note that the value update of Algorithm 1 is simply substituting the original state values with maximum action values since we are using greedy policy.

6.2.3 Convergence

The convergence can be proved with Theorem 4.4.1.1 and Theorem 4.4.2.1.

6.3 Policy Iteration

6.3.1 Main Idea

The **policy iteration** is based on the policy improvement theorem. Like value iteration, the policy iteration consists of two steps:

1. *Policy Evaluation*: For a given policy π_k, we calculate the corresponding state values by solving following Bellman equation:
$$v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k} \tag{6.4}$$
2. *Policy Improvement*: Using the computed state value v_k, we obtain the improved policy as
$$\pi_{k+1} = \arg\max_{\pi}(r_\pi + \gamma P_\pi v_{\pi_k}) \tag{6.5}$$
and here is where the policy improvement theorem is applied.

The difference is that the value iteration is that instead of taking a state value and computing the policy from it, the policy iteration takes a policy and compute the value corresponding to the policy.

Also, note that policy iteration has another iterative algorithm inside it: the policy evaluation uses iterative algorithm for solving the Bellman equation $v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}$.

6.3.2 Pseudocode

Algorithm 2: Policy Iteration Algorithm

Input : $p(s', r|s, a)$ ..... the dynamics of the system
π ..... initial guess

Output: v^*, π^* ..... optimal state value and optimal policy

repeat
  `// policy evaluation`
  $v' \leftarrow$ arbitrary initial guess
  repeat
    for each state $s \in \mathcal{S}$ **do**
      $v'(s) \leftarrow \sum_a \pi(a|s) \Big[\sum_r p(r|s,a) r + \gamma \sum_{s'} p(s'|s,a) v'(s') \Big]$
  until convergence of v'
  $v \leftarrow v'$
  `// policy improvement`
  for each state $s \in \mathcal{S}$ **do**
    for each action $a \in \mathcal{A}(s)$ **do**
      $q(s,a) \leftarrow \sum_r p(r|s,a) r + \gamma \sum_{s'} p(s'|s,a) v(s')$
    $a^* \leftarrow \arg\max_a q(s,a)$
    $\pi(a|s) \leftarrow \begin{cases} 1 & \text{if } a = a^* \\ 0 & \text{otherwise} \end{cases}$
until convergence of v
return v, π

6.3.3 Convergence

TODO: convergence proof of policy iteration theorem. This will be done after studying the operator version of Bellman Equation.

6.4 Truncated Policy Iteration

6.4.1 Main Idea

The **truncated policy iteration** is a more general algorithm including value iteration and policy iteration as its special cases. The algorithm has only one difference with policy iteration: in the policy evaluation step, the truncated policy iteration iterates only finite times, even when the convergence condition is not met. Truncated policy iteration algorithm is one of the instances of **general policy iteration**, which is an abstract concept of interleaving evaluation and improvement.

The perspective of looking at the truncated policy iteration as the generalization of policy iteration and value iteration can

be derived from the similiarity between value iteration and policy iteration. With the assumption that $v_{\pi}^{(0)} = v_0 = v_{\pi_0}$,

$$v_{\pi_1}^{(0)} = v_0 \tag{6.6}$$

$$\text{value iteration} \leftarrow v_1 \longleftarrow v_{\pi_1}^{(1)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(0)} \tag{6.7}$$

$$v_{\pi_1}^{(2)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(1)} \tag{6.8}$$

$$v_{\pi_1}^{(3)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(2)} \tag{6.9}$$

$$\vdots \tag{6.10}$$

$$\text{truncated policy iteration} \leftarrow \bar{v}_1 \longleftarrow v_{\pi_1}^{(j+1)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(j)} \tag{6.11}$$

$$\vdots \tag{6.12}$$

$$\text{policy iteration} \leftarrow v_{\pi_1} \longleftarrow v_{\pi_1}^{(\infty)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(\infty)} \tag{6.13}$$

$$\tag{6.14}$$

While value iteration computes only once and update the state value and policy iteration iterates until convergence and use it as state value of policy, the truncated policy iteration iterates only finite number of times.

6.4.2 Pseudocode

Algorithm 3: Truncated Policy Iteration Algorithm

Input : $p(s', r|s, a)$ ..... the dynamics of the system
π ..... initial guess

Output: v^*, π^* ..... optimal state value and optimal policy

repeat
  `// policy evaluation`
  $v' \leftarrow$ arbitrary initial guess
  for $k = 0, 1, \ldots, k_{\text{truncate}} - 1$ **do**
    for each state $s \in \mathcal{S}$ **do**
      $v^{(k+1)}(s) \leftarrow \sum_a \pi(a|s) \Big[\sum_r p(r|s, a) r + \gamma \sum_{s'} p(s'|s, a) v^{(k)}(s') \Big]$
  $v \leftarrow v^{(k_{\text{truncate}})}$
  `// policy improvement`
  for each state $s \in \mathcal{S}$ **do**
    for each action $a \in \mathcal{A}(s)$ **do**
      $q(s, a) \leftarrow \sum_r p(r|s, a) r + \gamma \sum_{s'} p(s'|s, a) v(s')$
    $a^* \leftarrow \arg\max_a q(s, a)$
    $\pi(a|s) \leftarrow \begin{cases} 1 & \text{if } a = a^* \\ 0 & \text{otherwise} \end{cases}$
until convergence of v
return v, π

6.4.3 Convergence

TODO: Convergence proof of truncated policy iteration.

Chapter 7

Monte Carlo Methods

7.1 Introduction

This chapter introduces a **Monte Carlo methods** for reinforcement learning. These algorithms all starts from trying to obtain the optimal policy by solving the Bellman optimality equation. The difference of pure dynamic programming algorithms and Monte Carlo methods is that the latter do not require a perfect dynamics of the model, i.e., it is *model-free*.

Then, how do we get the optimal policy without a perfect model? We use the data that the model gives us. Using the data, we can convert the policy iteration algorithm 2 to the model-free version. Specifically, we estimate the action value $q(s,a) = \mathbb{E}[G_t|S_t = a, A_t = a]$ using Monte Carlo sampling.

7.2 Monte Carlo Basic

7.2.1 Main Idea

The **Monte Carlo basic** is the simplest Monte Carlo method algorithm. Suppose that we run the agent starting from state-action pair (s,a), following policy π, and let $g_\pi^{(i)}(s,a)$ be the return of the episode. Then, we can approximate the action value $q_\pi(s,a)$ as following:

$$q_\pi(s,a) = \mathbb{E}[G_t|S_t = s, A_t = a] \approx \frac{1}{n}\sum_{i=1}^{n} g_\pi^{(i)}(s,a). \tag{7.1}$$

If the number n is sufficiently large, the approximation will be sufficiently accurate. This can be guaranteed due to the law of large numbers.

7.2.2 Pseudocode

Algorithm 4: Monte Carlo Basic

Input : π_0 ..... initial guess
Output: π^* ..... optimal policy

repeat
  foreach $s \in \mathcal{S}$ **do**
    foreach $a \in \mathcal{A}(s)$ **do**
      `// policy evaluation`
      collect sufficiently many episodes starting from (s, a), following policy π_k
      $q_{\pi_k}(s, a) \leftarrow$ the average return of all episodes starting from (s, a)
  `// policy improvement`
  $\pi_{k+1}(a|s) = \begin{cases} 1 & \text{if } a = \arg\max_{a'} q_{\pi_k}(s, a') \\ 0 & \text{otherwise} \end{cases}$
until convergence of π
return π

7.2.3 Convergence

7.2.4 Pros and Cons

Cons:

- Very inefficient sample usage. Due to this property, MC-Basic is not used in practical. To deal with this, we use MC with exploring start, which is presented in next section.

7.3 Monte Carlo with Exploring Starts

7.3.1 Main Idea

The MC-Basic algorithms is not sample-efficient, since it only updates only q value of one state-action pair each episode. To deal with this low efficiency, we use all the *visit* to the state-action pair in each episode.

Suppose that the agent has gone through following trajectory:

$$s_1 \xrightarrow{a_1} s_2 \xrightarrow{a_2} s_3 \xrightarrow{a_3} s_4 \xrightarrow{a_4} \dots \tag{7.2}$$

To increase the sample efficiency, we use the visits to the state-action pairs in the trajectory. That is, using the above episode, we not only use (s_1, a_1) for estimating $q(s_1, a_1)$ but use (s_2, a_2) for estimating $q(s_2, a_2)$, and so on. There are two different ways to do this:

- The first way is to only use the state-action pairs when it is first encountered. This is called *first-visit method*;
- The second way is to use all the visit to the state-action pairs. This is called *every-visit method*.

Also, we update the estimate of q values after each episode ends. This falls into the scope of *generalized policy iteration*, since we are using the value approximation even it is not so accurate.

7.3.2 Pseudocode

Algorithm 5: Monte Carlo with Exploring Starts (every-visit)

Input : π_0 ..... initial policy
Output: π^* ..... optimal policy

`// initialize`
foreach valid state-action pair (s, a) **do**
  $\pi \leftarrow \pi_0$
  $q(s, a) \leftarrow$ initial value
  `// holds the sum of returns of (sub)episode starting from` (s, a)
  $\text{Ret}(s, a) \leftarrow 0$
  `// counts the number of visits of` (s, a)
  $\text{N}(s, a) \leftarrow 0$
repeat
  select a starting valid state-action pair (s, a), ensuring that all the pairs can be possibly selected
  generate an episode starting from (s, a), following current policy π: $s_0, a_0, r_1, s_1, a_1, \ldots, s_{T-1}, a_{T-1}, r_T$
  $G \leftarrow 0$
  for $t = T-1, T-2, \ldots, 0$ **do**
    $G \leftarrow r_{t+1} + \gamma G$
    $\text{Ret}(s_t, a_t) \leftarrow \text{Ret}(s_t, a_t) + G$
    $\text{N}(s_t, a_t) \leftarrow \text{N}(s_t, a_t) + 1$
    `// policy evaluation`
    $q(s_t, a_t) \leftarrow \frac{\text{Ret}(s_t, a_t)}{\text{N}(s_t, a_t)}$
    `// policy improvement`
    $\pi_{k+1}(a_t|s_t) = \begin{cases} 1 & \text{if } a = \arg\max_{a'} q_{\pi_k}(s, a') \\ 0 & \text{otherwise} \end{cases}$
until convergence

7.3.3 Implementation Details

The exploring-start condition can be achieved by random start state.

7.3.4 Pros and Cons

Pros:

- Better sample efficiency then MC-Basic;
- All state-action pairs can be visited due to the exploring-start, and this makes the estimation more accurate.

Cons:

- Only applicable to episodic tasks, which is a common drawback for all MC methods. This property leads to following problem, too;
- High variance since the return holds all the noises and randomnesses, and the return is used for update;
- If the exploring start condition cannot be satisfied, the MC with Exploring Starts cannot be used. This is common in real-world problems. To deal with this, another algorithms without exploring starts will be introduced next section;

7.4 Monte Carlo without Exploring Starts

7.4.1 Main Idea

To deal with the exploring starts, we use ε-greedy function for exploring. That is, instead of using the greedy policy for training, we train the ε-greedy policy. This little probability of choosing non-greedy action is required, since the greedy action we have depends only on current informations. That is, there can be a better choice of action selection, since the action values are not perfectly known. Actually, balancing the choice of greedy and non-greedy action is a very important problem in reinforcement learning. This problem is called the problem of **exploitation** and **exploration**.

Remark 7.4.1.1 (Problem of Exploitation and Exploration)

To **exploit** is to use the current information and select the greedy action. To **explore** is to select the action other than greedy action. Since the true state values cannot be known in training, the action that looks greedy may not lead to the highest total reward. If the agent exploits too much, the agent may find only suboptimal policies. If the agent explores too much, the agent won't be able to find the optimal policy. As aforementioned, the balance between exploitation and exploration is critical.

7.4.2 Pseudocode

Algorithm 6: Monte Carlo with ε-greedy policy

Input : π_0 ..... initial policy
$\varepsilon \in (0, 1)$ ..... probability of exploration

Output: π^* ..... optimal policy

`// initialize`

foreach valid state-action pair (s, a) **do**
  $\pi \leftarrow \pi_0$
  $q(s, a) \leftarrow$ initial value
  `// holds the sum of returns of (sub)episode starting from` (s, a)
  $\mathrm{Ret}(s, a) \leftarrow 0$
  `// counts the number of visits of` (s, a)
  $\mathrm{N}(s, a) \leftarrow 0$

repeat
  select a starting valid state-action pair (s, a)
  generate an episode starting from (s, a), following current policy π: $s_0, a_0, r_1, s_1, a_1, \ldots, s_{T-1}, a_{T-1}, r_T$
  $G \leftarrow 0$
  for $t = T - 1, T - 2, \ldots, 0$ **do**
    $G \leftarrow r_{t+1} + \gamma G$
    $\mathrm{Ret}(s_t, a_t) \leftarrow \mathrm{Ret}(s_t, a_t) + G$
    $\mathrm{N}(s_t, a_t) \leftarrow \mathrm{N}(s_t, a_t) + 1$
    `// policy evaluation`
    $q(s_t, a_t) \leftarrow \frac{\mathrm{Ret}(s_t, a_t)}{\mathrm{N}(s_t, a_t)}$
    `// policy improvement`
    $\pi_{k+1}(a_t|s_t) = \begin{cases} 1 - \varepsilon + \frac{\varepsilon}{|\mathcal{A}(s)|} & \text{if } a = \arg\max_{a'} q(s, a) \\ \frac{\varepsilon}{|\mathcal{A}(s)|} & \text{otherwise} \end{cases}$

until convergence

7.4.3 Implementation Details

The problem with ε-greedy policy is that it even if the trained policy is optimal in the spaces of ε-greedy policies, the policy still has a probability of exploration. To deal with this, ε-greedy policy with GLIE (Greedy in the Limit with Infinite Exploration).

Definition 7.4.3.1 (GLIE Condition)

When following two conditions are satisfied, the policy is called GLIE.

- Infinite Exploration: Every state-action pair must be visited and explored infinitely many times as the training steps go to infinity (i.e., $t \to \infty$).
- Limit Greediness: As the learning agent experiences more of the environment, the policy must gradually shift from random exploration to exploiting what it knows, eventually converging entirely to a greedy (optimal) policy.

The most simple way to establish this condition is to use ε-greedy policy with decreasing ε. We can decay the ε linearly, exponentailly or whatever. TODO: search for several GLIE policies.

According to the approximation theories in Chapter 5, we can also estimate the state values using the stochastic approximation, without using the $\text{Ret}(s, a)$ and $\text{N}(s, a)$ in Algorithm 6. The mainly used approximation is to update the action value as

$$q(s_t, a_t) \leftarrow q(s_t, a_t) + \alpha(G_t - q(s_t, a_t)) \tag{7.3}$$

where α is a fixed constant.

7.4.4 Pros and Cons

Pros:

- Better sample efficiency then MC-Basic;
- Does not require exploring starts, and still explores sufficiently;

Cons:

- Only applicable to episodic tasks, which is a common drawback for all MC methods. This property leads to following problem, too;
- High variance since the return holds all the noises and randomnesses, and that return is used for update;

Chapter 8

Temporal Difference Methods

8.1 Introduction

This chapter presents the temporal-difference (TD) methods for reinforcement learning. The main difference of MC and TD is that TD learning is incremental, while MC is not. That is, TD methods are appliable on continuous tasks, unlike MC method.

8.2 TD Learning of State Values

The general algorithm of TD method with state value is given in this section. In practice, we use TD method for action values, but the one given here is a foundation of them.

8.3 Sarsa

Now, we introduce the most basic TD-algorithm, the **Sarsa**. Its name originates from the tuple (s, a, r, s', a') it uses.

In the last section, the TD algorithm of learning state value was mainly discussed. Here, we directly learn the action values instead of state values, since it can be combined with a policy improvement step to learn optimal policies.

8.3.1 Main Idea

Suppose we have trajectory generated from policy π: $(s_0, a_0, r_1, s_1, a_1, \dots, s_t, a_t, r_{t+1}, s+t+1, a_{t+1}, \dots)$. Note that the trajectory does not have to end. We use following TD-update every step for estimating the action values:

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) + \alpha_t(s_t, a_t)\Big[r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1}) - q_t(s_t, a_t)\Big] \tag{8.1}$$

$$q_{t+1}(s, a) = q_t(s, a), \text{ for all } (s, a) \neq (s_t, a_t) \tag{8.2}$$

In fact, the Sarsa solves following Bellman equation in a stochastic approximation manner.

$$q_\pi(s, a) = \mathbb{E}[R + \gamma q_\pi(S', A')|s, a] \text{ for all } (s, a) \tag{8.3}$$

Equation 8.3 is the Bellman equation about action values. The proof is given below.

Proof. Note that

$$p(s', a'|s, a) = p(s'|s, a)p(a'|s', s, a) \tag{8.4}$$
$$= p(s'|s, a)p(a'|s') \text{ (due to the Markov property)} \tag{8.5}$$
$$= p(s'|s, a)\pi(a'|s'). \tag{8.6}$$

Using this, the equation of Theorem 3.4.2.1 can be written as

$$q_\pi(s, a) = \sum_r rp(r|s, a) + \gamma \sum_{s',a'} q_\pi(s', a')p(s', a'|s, a) \tag{8.7}$$

□

8.3.2 Pseudocode

Algorithm 7: Sarsa

Input : $\alpha_t(s, a) \in (0, 1]$ ..... step size parameter satisfying RM condition

Output: q_* ..... the optimal policy

```
// initialize
```
foreach $(s, a) \in \mathcal{S} \times \mathcal{A}$ **do**

  $q(s, a) \leftarrow \begin{cases} \text{arbitrary}, & \text{if } s \in \mathcal{S}^+ \text{ and } a \in \mathcal{A}(s) \\ 0 & \text{if } s \in \mathcal{S} \setminus \mathcal{S}^+ \end{cases}$

$\pi \leftarrow$ policy derived from q (e.g. ε-greedy, softmax, GLIE)

foreach episodes **do**

  initialize s

  choose action a from s using policy π

  repeat

    take action a, observe r, s'

    choose action a' from s' using policy derived from π

    `// update q value`

    $q(s, a) \leftarrow q(s, a) + \alpha_t(s, a)\big[r + \gamma q(s', a') - q(s, a)\big]$

    `// update policy`

    update π according to $q(s, a)$ $s \leftarrow s'$

    $a \leftarrow a'$

  until s is in terminal state

return $q \simeq q_*$

8.3.3 Implementation Detail

I usually intialize q value without taking account the terminal states. While traning the agent, when the agent meets terminal state, I just check it immediatedly and set the $q_t(s_{t+1}, a_{t+1}) = 0$. Actually I don't even change the q value of terminal state to 0. I just leave it untouched.

8.3.4 Convergence

TODO: Convergence of Sarsa

8.3.5 Pros and Cons

Pros:

- It learns safer paths by accounting for penalties from exploratory moves. This can be checked with cliff walking of [Sut20], with comparison to Q-Learning.

Cons:

- It learns suboptimal policies first, due to the behavior of search for safe paths in the initial training process.
- The variance can be high, since in the initial training process the q values are very noisy and inaccurate, and the agent use the q value of it directly in the update. This can be dealt with the Expected Sarsa 8.5.

8.4 Q-Learning

In this section we learn another TD-learning algorithm, the **Q-Learning**. Unlike Sarsa, Q-Learning bootstraps the q value using the maximum action value of current state.

8.4.1 Main Idea

The Q-Learning uses following update equation for action values:

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) + \alpha_t(s_t, a_t)\Big[r_{t+1} + \gamma \max_{a \in \mathcal{A}(s_{t+1}, a)} q_t(s_{t+1}, a) - q_t(s_t, a_t)\Big] \tag{8.8}$$

$$q_{t+1}(s, a) = q_t(s, a), \text{ for all } (s, a) \neq (s_t, a_t) \tag{8.9}$$

The Q-Learning solves following Bellman optimality equation:

$$q(s, a) = \mathbb{E}\Big[R_{t+1} + \gamma \max_a q(S_{t+1}, a) | S_t = s, A_t = a\Big] \tag{8.10}$$

Proof. Check for [Zha25] pg.140. □

Remark 8.4.1.1 (On-policy and Off-policy)

As one can see, the Sarsa solves Bellman equation and then make policy update, but Q-Learning directly learns the optimal policy. The **behavior policy**, is used to make the trajectory. The **target policy** is the policy that learns the optimal policy. When the behavior policy and target policy equals, the algorithms is called **on-policy** algorithm. If not, it is called **off-policy**. Off policy can separate the exploration with target policy. The target policy only has to care about choosing the best action in current situation. Since the off-policy uses policy other than its target policy, it requires some techniques such as **importance sampling** and **tree-backup** algorithms. These will be covered later.

The on-policy algorithms cannot be implemented as an **offline** fashion, since it requires real-time interaction with environment. The off-policy algorithms, in contrast, can be implemented as an offline or online fashion.

8.4.2 Pseudocode

Algorithm 8: Q-Learning

Input : $\alpha_t(s,a) \in (0,1]$ ..... step size parameter satisfying RM condition
b ..... behavior policy
π ..... target policy

Output: q_* ..... the optimal policy

`// initialize`

foreach $(s,a) \in \mathcal{S} \times \mathcal{A}$ **do**

$$q(s,a) \leftarrow \begin{cases} \text{arbitrary}, & \text{if } s \text{ is not in terminal states} \\ 0 & \text{if } s \text{ is in terminal states} \end{cases}$$

$s \leftarrow$ initialize

foreach episode **do**

  repeat

    choose action a from s using behavior policy b

    observe state s' and reward r

    `// update q value`

    $q(s,a) \leftarrow q(s,a) + \alpha_t(s,a)\left[r + \gamma \max_a q(s',a) - q(s,a)\right]$ `// update target policy`

    update target policy according to q

    $s \leftarrow s'$

  until s is not terminal state

8.4.3 Implementation Detail

The behavior policy can be any policy that explores well, but in practice we use decaying ε-greedy due to the convergence rate. (I guess...)

8.4.4 Convergence

TODO: Convergence of Q-Learning

8.4.5 Pros and Cons

Pros:

- It is off-policy. Whatever the behavior policy does, the target policy learns the optimal policy.
- Does not fears getting penalty, as opposed to Sarsa. This can be checked in [Sut20], cliff walking.

Cons:

- Can suffer from **maximization bias**, since the Q-Learning uses maximum q value for bootstrapping. That is, the agent keeps overestimate the q values, which can lead to slow convergence. For more information, see [Sut20]. To cope with this, we can use **Double Q-Learning**.

8.5 Expected Sarsa

Chapter 9

Function Approximation Methods

9.1 Introduction

In this chapter, value function approximation methods will be covered. Unlike the tabular methods, the approximate method tries to approximate the state value or action value of state-action pairs, using a function. This solves the curse of dimensionality, which is a critical problem for tabular methods.

But there are tradeoffs, too. For example, the convergence property breaks down easily. When off-policy, function approximation, and bootstrapping meets together, (**the deadly triad**, [Sut20]), the algorithm lost its convergence property and fails to find optimal policy. There are various ways to solve this problem. Some solve this with practical methods (replay buffer, PER, double learning, etc.), and some solve this theoretically (GTD2, Emphatic-TD, etc.). I'll try to cover both of them in this note.

9.2 TD Learning of State Values based on Function Approximation

Methods of estimating the state value with a function approximation is introduced in this section.

9.2.1 Objective Function

Let $v_\pi(s)$ be the true state value function and let $\hat{v}_\pi(s, w)$ be the approximation of it with a parameter w. To find the optimal parameter w which best approximates the true state value function, we minimize the objective function. In particuluar, the objective function is given as

$$J(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S, w))^2], \tag{9.1}$$

where S is a random variable of state space. Then what is the distribution of a random variable S? We can use *stationary distribution* of Markov Process. Stationary distribution of a Markov Process describes the long-term behavior of a Markov decision process. That is, the probability of agent being located at certain state after a long execution can be described by this distribution.

Let $d_\pi(s)$ be the stationary distribution of the Markov process under policy π. Then, the objective functon can be written

as

$$J(w) = \sum_{s \in \mathcal{S}} d_\pi(s)(v_\pi(s) - \hat{v}(s, w))^2 \tag{9.2}$$

TODO: Why stationary distribution? + Tsitsiklis and van Roy + Chapter 6

9.3 Deep Q-Learning

9.3.1 Main Idea

The **Deep Q-Learning** [Mni+13] is one of the earliest and successful deep reinforcement learning algorithms. Suprisingly, DQN algorithm are not guaranteed to converge, since the deadly triad perfectly fits into the algorithms' condition: it bootstraps, it is off-policy, and it uses function approximation. But there are practical methods for calming down this unstability. But still note that the algorithm is not guaranteed to converge.

9.3.2 Objective Function

The objective function which DQN minimizes is given as following:

$$J = \mathbb{E}\Big[\Big(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w) - \hat{q}(S, A, w)\Big)^2\Big], \tag{9.3}$$

where $\hat{q}$ is the approximation function of action values and (S, A, R, S') are random variables denoting state, action, reward and next state, resepectively. Note that the squared term is a Bellman optimality error.

Now, we can caculate the gradient and minimize the loss with gradient descents... but take a look. Unfortunately, the max operation is not differentiable. To resolve this, we freeze the parameter of a network inside the max operation for a short time. Specifically, we introduce two networks: *target network and main network*. Using the main network $\hat{q}(s, a, w)$ and $\hat{q}(s, a, w_T)$, we can re-write the objective function as following:

$$J = \mathbb{E}\Big[\Big(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w_T) - \hat{q}(S, A, w)\Big)^2\Big], \tag{9.4}$$

and the gradient can be computed about w. The parameters of target network and main network are synchronized regularly.

There is one more technique required for DQN to be trained. The *experience replay* collects the (s, a, r, s') pair, and then samples the experiences with uniform random distrubution. Why is this technique required?

- [Zha25] says: Compare equation 9.4 and 9.2. Since we don't know the probability distribution of (S, A, R, S'), we just simply set it as uniform distrubution. However, the state-action pairs are are strongly correlated, and not uniformly distributed in single trajectory. To break the correlation, we uniformly draw samples from the replay buffer.

Also, since we can reuse the experiences, the sample efficiency naturally increases.

9.3.3 Pseudocode

Algorithm 9: Deep Q-Learning

Input : b ..... behavior policy
C ..... sync rate

Output: $\hat{q}(s, a, w)$ ..... approximated action values

store the samples generated by b in a replay buffer $\mathcal{B} = \{(s, a, r, s')\}$
foreach iteration **do**
  uniformly draw a mini batch B of samples from $\mathcal{B}$
  $L \leftarrow 0$
  foreach sample $(s, a, r, s') \in B$ **do**
    $y_T \leftarrow r + \gamma \max_{a \in \mathcal{A}(s')} \hat{q}(s', a, w_T)$
    $L \leftarrow L + \Big(y_T - \hat{q}(s, a, w)\Big)^2$
  minimize L by updating parameter w
  set $w_t = w$ every C iterations
return $\hat{q}$

9.3.4 Implementation Details

Since the original DQN algorithm is highly unstable, there are several techniques to stabilize the algorithm.

- Priortized experience replay (PER)
- Double DQN
- Dueling DQN
- Distributional DQN

9.3.5 Pros and Cons

Pros:

-

Cons:

- Catastrophic forgetting: This is a common problems in neural networks.
- No convergence guaranteed: Note that DQN satisfies the deadly triad condition. And also, the algorithm does not even use the true gradient, since the target network is freezed for a moment. This is called a **semi-gradient** method, and no convergence is guaranteed for this method. For more information see [Sut20].

Chapter 10

Policy-based Methods

10.1 Introduction

Policy gradient method, unlike value-based methods, tries to predict the optimal policy itself, instead of solving the Bellman equation. It has many advantages over value-based methods; It is more efficient in handling large state-action spaces, and has stronger generalization abilities.

10.2 REINFORCE

10.2.1 Main Idea

REINFORCE is the most basic form of policy gradient method. It's the vanilla policy gradient method.

10.2.2 Objective Function

REINFORCE uses following objective function;

$$J(\pi_{\boldsymbol{\theta}}) = \mathbb{E}_{\tau\sim\pi_{\boldsymbol{\theta}}}[G_t] = \mathbb{E}_{\tau\sim\pi_{\boldsymbol{\theta}}}\left[\sum_{t=0}^{T-1}\gamma^t r_{t+1}\right]. \tag{10.1}$$

We use some tricks to compute the gradient of $J(\pi_{\boldsymbol{\theta}})$ Then we get

$$\nabla_{\boldsymbol{\theta}} J(\pi_{\boldsymbol{\theta}}) = \mathbb{E}_{\tau\sim\pi_{\boldsymbol{\theta}}}\left[\sum_{t=0}^{T-1} G_t \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(a_t|s_t)\right]. \tag{10.2}$$

Now we can use this to update the parameter $\boldsymbol{\theta}$. Since we cannot compute the exact expectation, we use Monte Carlo sampling to approximate $\nabla_{\boldsymbol{\theta}} J(\pi_{\boldsymbol{\theta}})$ as following:

$$\nabla_{\boldsymbol{\theta}} J(\pi_{\boldsymbol{\theta}}) \approx \sum_{t=0}^{T-1} G_t \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(a_t|s_t) \tag{10.3}$$

10.2.3 Pseudocode

Algorithm 10: REINFORCE

Input : $\boldsymbol{\theta}$ ..... initial parameter
γ ..... discount rate
α ..... learning rate

Output: $\pi_{\boldsymbol{\theta}}$ ..... learned policy

initialize policy $\pi_{\boldsymbol{\theta}}$
repeat
  generate episode $\{s_0, a_0, r_1, \ldots, s_{T-1}, a_{T-1}, r_T\}$ following $\pi_{\boldsymbol{\theta}}$
  for $t = 0, 1, \ldots, T-1$ **do**
    `// compute return-to-go`
    $G_t = \sum_{k=t+1}^{T} \gamma^{k-t-1} r_k$
    `// policy update`
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(a_t|s_t) G_t$
until convergence
return $\pi_{\boldsymbol{\theta}}$

10.2.4 Implementation Notes

- When computing return-to-go, we can iterate backward from the terminal time to start, since $G_t \leftarrow \gamma G_{t+1} + r_{t+1}$ where $t = 0, 1, 2, \ldots, T-1$.

10.2.5 Pros and Cons

Pros:

- No need for model;
- Can cover both discrete and continuous actions;

Cons:

- Large variance;
- May converge to suboptimal policy;

Chapter 11

Model-based Methods

Chapter 12

Comparisons of Algorithms

Part III

Appendices

Chapter 13

Probability Theory

13.1 Introduction

This section introduces a theory of measure-theoretic probability, based on [BZ07].

13.2 Measure-Theoretic Probability

13.2.1 Basic Definitions

Definition 13.2.1.1 (σ-field)

Let Ω be a non-empty set. A σ-field on Ω is a family of subsets of Ω such that

1. the empty set $\emptyset \in \mathcal{F}$;
2. if $A \in \mathcal{F}$ then $\Omega \setminus A \in \mathcal{F}$;
3. if $A_1, A_2, \dots$ is a sequence of sets in $\mathcal{F}$, then $A_1 \cup A_2 \cup \dots \in \mathcal{F}$

Definition 13.2.1.2 (Probability Measure)

Let $\mathcal{F}$ be a σ-field on Ω. A **probability measure** P is a function $P : \mathcal{F} \to [0, 1]$ such that

1. $P(\Omega) = 1$;
2. if $A_1, A_2, \dots$ are pairwise disjoint sets ($A_i \cap A_j = \emptyset$ for all $i \neq j$) beloning to $\mathcal{F}$, then

$$P(A_1 \cup A_2 \cup \dots) = P(A_1) + P(A_2) + \dots. \tag{13.1}$$

The triple $(\Omega, \mathcal{F}, P)$ is called a **probability space**. The sets belonging to $\mathcal{F}$ are called **events**. An event A is said to occur **almost surely** (a.s.) whenever $P(A) = 1$.

Definition 13.2.1.3 (Random Variable)

Let $\mathcal{F}$ be a σ-field on Ω. Then a function $\xi : \Omega \to \mathbb{R}$ is said to be $\mathcal{F}$-measurable if the following is satisfied:

$$\{\xi \in B\} \in \mathcal{F} \text{ for every Borel set } B \in \mathcal{B}(\mathbb{R}). \tag{13.2}$$

where **Borel sets** refers to the elements of $\mathcal{B}(\mathbb{R})$ which is the smallest σ-field containing all intervals in $\mathbb{R}$. If the triple $(\Omega, \mathcal{F}, P)$ is a probability space, then such a function ξ is called a **random variable**.

Remark 13.2.1.1 (Short-hand Notation for Events)

The set of events such as $\{\xi \in B\}$ represents the set $\{\omega \in \Omega | \xi(\omega) \in B\}$.

Chapter 14

Linear Algebra

14.1 Introduction

A short useful theorems of linear algebra.

14.2 Geshgorin Circle Theorem

Theorem 14.2.0.1 (Geshgorin Circle Theorem)

Bibliography

[Axl20] Sheldon Jay Axler. *Measure, integration and real analysis*. Graduate texts in mathematics 282. Literaturverzeichnis: Seite 402. Cham, Switzerland: Springer Open, 2020. 411 pp. ISBN: 9783030331429.

[BZ07] Zdzisław Brzeźniak and Tomasz Zastawniak. *Basic stochastic processes. A course through exercises*. 10. print. Springer undergraduate mathematics series. London: Springer, 2007. 225 pp. ISBN: 3540761756.

[Gra20] Laura Graesser. *Foundations of deep reinforcement learning. Theory and practice in Python*. Ed. by Wah Loon Keng. Addison Wesley data & analytics series. Includes bibliographical references and index. Boston: Addison-Wesley, 2020. 379 pp. ISBN: 0135172381.

[Mni+13] Volodymyr Mnih et al. "Playing Atari with Deep Reinforcement Learning". In: *CoRR* abs/1312.5602 (2013). DOI: 10.48550/arxiv.1312.5602. arXiv: 1312.5602. URL: http://arxiv.org/abs/1312.5602.

[Sut20] Richard S. Sutton. *Reinforcement learning. An introduction*. Ed. by Andrew Barto. Second edition. Adaptive computation and machine learning. Hier auch später erschienene, unveränderte Nachdrucke. Cambridge, Massachusetts: The MIT Press, 2020. 526 pp. ISBN: 9780262039246.

[Zha25] Shiyu Zhao. *Mathematical foundations of reinforcement learning*. [Tsinghua]: Tsinghua University Press, 2025. 1275 pp. ISBN: 9789819739448.